

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is light green. They are positioned diagonally, with the blue one partially covering the green one.

Algoritma & Struktur Data

Week 13 - Graph



Last Week Review

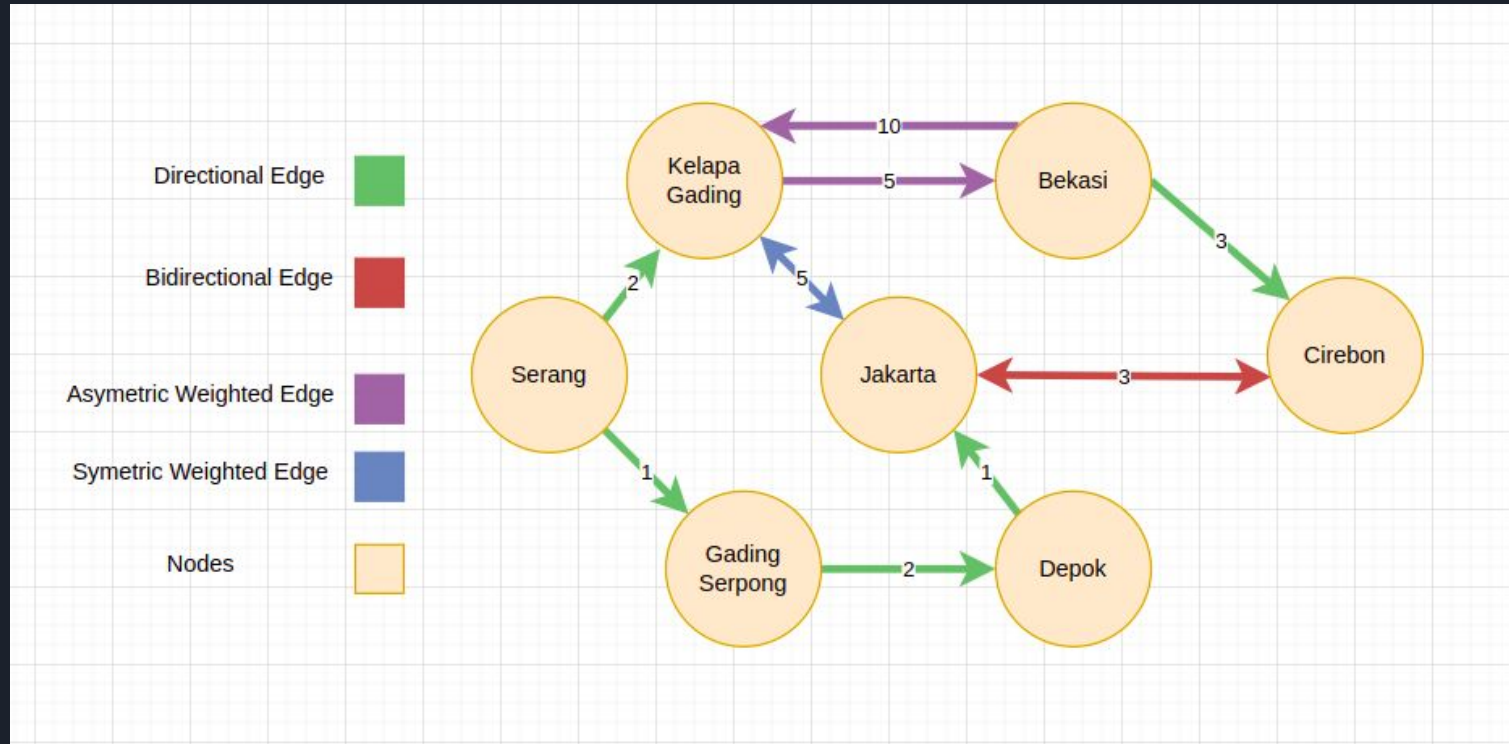
- Disjoint Set
 - A set which each member does not intersect with other set.
 - Several Main method:
 - $\text{make_set}(x) \leftarrow$ it will create a set with x as it's only member and a set representative at the same time
 - $\text{Find}(x) \leftarrow$ it will return the representative member of the set.
 - $\text{union}(a,b) \leftarrow$ will merge set A with set B , which the representative will be decided based on either:
 - Ranks
 - Size



Graph

- Data structure which contains a collection of a set of nodes (vertices/vertex) and a set of edges (connecting line) between the nodes.
- There are two type of Path
 - Bidirectional Path: Graph Path with no specific direction (can move back and forth between the nodes)
 - Directional Path: Graph Path with a specific direction
- There are two type of graph
 - Weighted Graph: Graph with each path may have different value. For example, we can represent the amount of time needed to travel between one city and another as a weight value.
 - Jakarta $\rightarrow$ Kelapa gading (1 Hour)
 - Kelapa gading $\rightarrow$ Jakarta (2 Hour) (It is possible to have asymmetric value on bidirectional Path.
 - Unweighted Graph: Graph that does not have any value on it's path.
- Formal Definition of the graphs would be:
 - $G=(V,E)$
 - $V(G)$: set of vertices (non-empty and finite)
 - $E(G)$: set of edges (Pairs of vertices)

Graph Anatomy

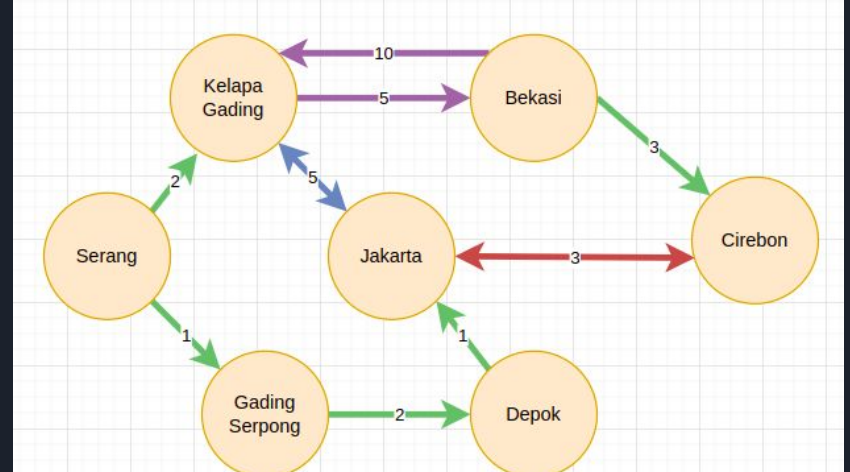
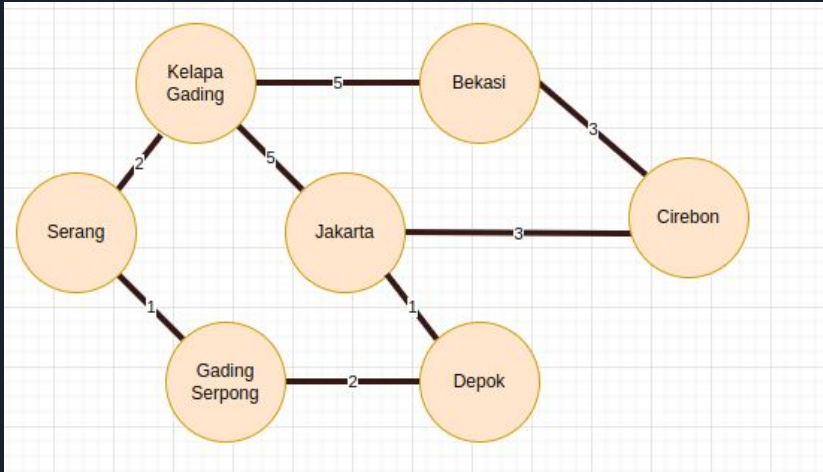


$V(\text{Graph1}) = \{\text{Serang, Kelapa Gading, Jakarta, Depok, Bekasi, Cirebon, Gadin Serpong}\}$

$E(\text{Graph1}) = \{(\text{serang, Gading serpong}), (\text{Serang, Kelapa gading}), (\text{Jakarta, Kelapa Gading}), (\text{Kelapa Gading, Jakarta}), \dots, \text{dst}\}$

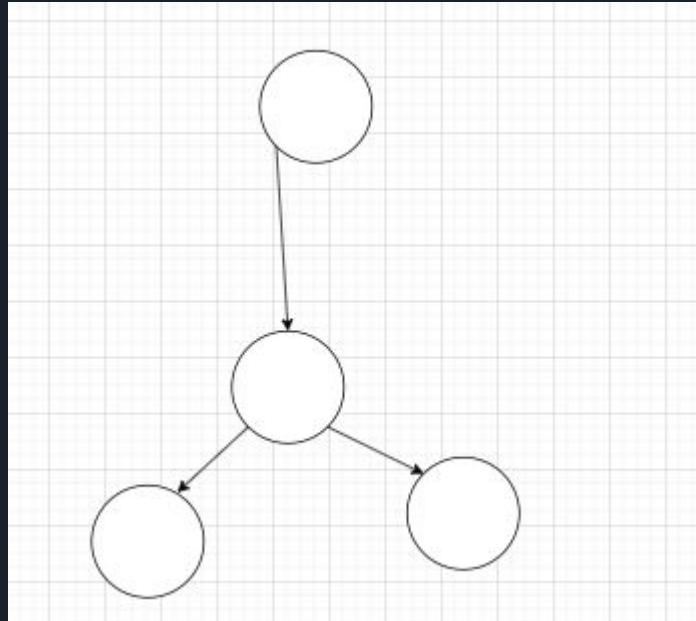
Directed Graph Vs Undirected Graph

- Undirected means no Path in the graph have any direction, While the directed have a specific direction on it's edge.
- Vertex Order Matters in a directed graph.



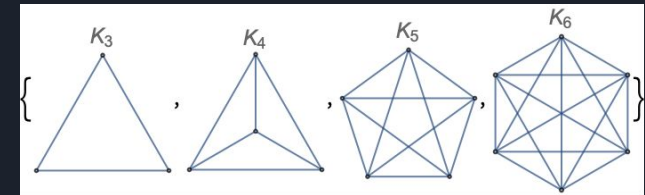
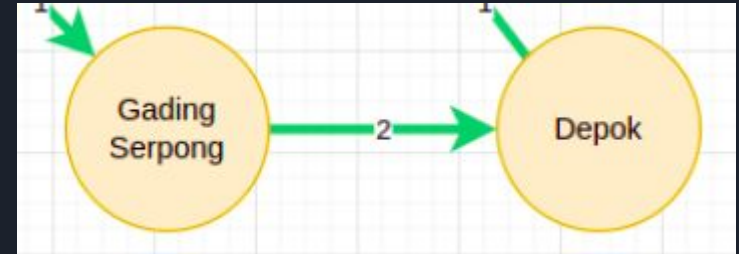
Tree & Graph

- Tree is a directed graph with each node only connected through one edge only.



Graph Terminology

- Adjacent Nodes: The nodes that is connected by an edge is called Adjacent Nodes (Neighbouring Nodes)
 - Gading Serpong is adjacent to Depok
 - Depok is adjacent from Gading Serpong
- Path: A sequence of vertices that helps two nodes in graph connected.
- Complete Graph: Graph where every node is adjacent toward every other nodes.
 - Number of possible pairs given N?
 - $N * (N-1)$
 - What about the number edges?
 - $(N*(N-1))/2$
 - How about how many possible path given N vertices?
 - THINK ABOUT IT :D



Cyclic VS Acyclic Graph

- Cyclic:
 - Graph where it has at least one path that make a cycle. It can be detected if we have a path where we can start and arrive at the same node.
- Acyclic:
 - It is the opposite of cyclic graph. The trivial example would be a Tree.

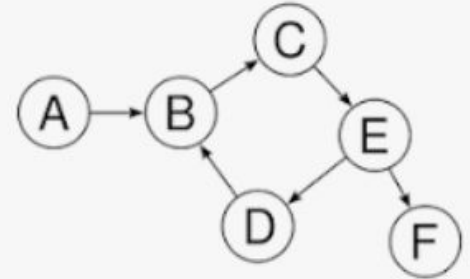


Figure 5 : Cyclic Graph

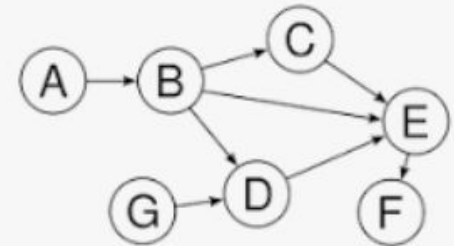
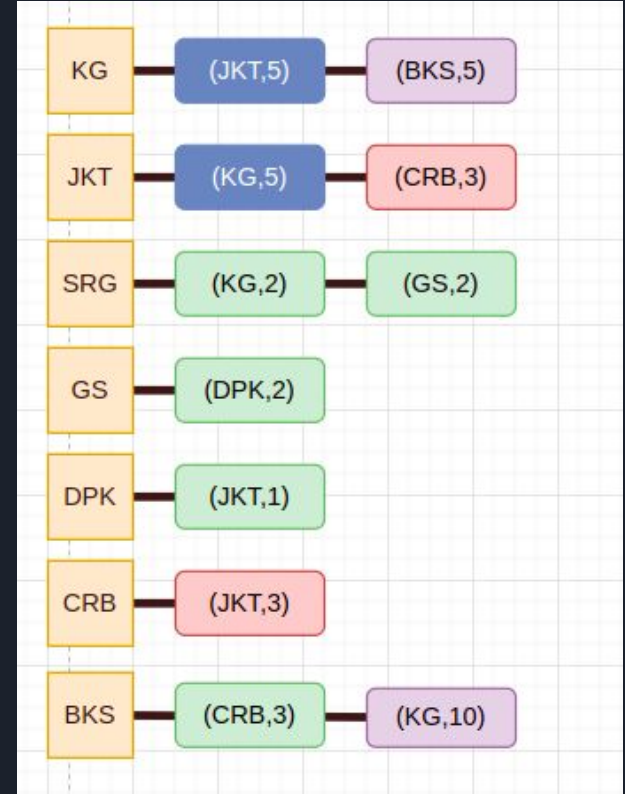
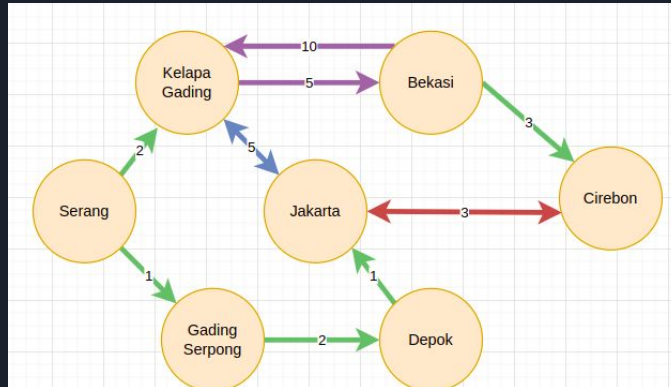


Figure 6 : Acyclic Graph

Graph Representation (Adjacency List)

- Adjacency List
 - Creating array of List, the array size will be exactly same with the number of vertex.
 - List[i] represent the connection between vertex i and it's adjacent nodes. We can even include the weight if we represent each item as a pair e.g: pair(node_index,weight)
 - Harder to Implement, Save more memory (suitable for large Graph)



Graph Representation (Adjacency List) (Code)

```
struct node {
    int vertex;
    struct node* next;
};

struct Graph {
    int N;
    struct node** adjLists;
};
```

```
struct node* createNode(int v) {
    struct node* newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}
```

```
void addEdge(struct Graph* graph, int s, int d) {
    struct node* newNode = createNode(d);
    newNode->next = graph->adjLists[s];
    graph->adjLists[s] = newNode;

    newNode = createNode(s);
    newNode->next = graph->adjLists[d];
    graph->adjLists[d] = newNode;
}
```

```
struct Graph* createAGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->N = vertices;

    graph->adjLists = malloc(vertices * sizeof(struct node));

    int i;
    for (i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;

    return graph;
}
```

```
void printGraph(struct Graph* graph) {
    int v;
    for (v = 0; v < graph->numVertices; v++) {
        struct node* temp = graph->adjLists[v];
        printf("\n Vertex %d\n: ", v);
        while (temp) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("\n");
    }
}
```

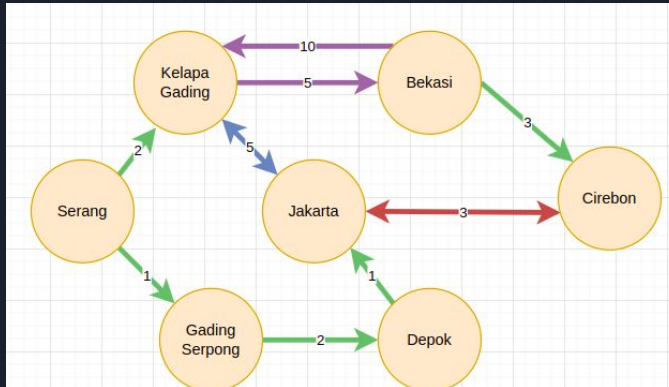
```
int main() {
    struct Graph* graph = createAGraph(4);
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 0, 3);
    addEdge(graph, 1, 2);

    printGraph(graph);

    return 0;
}
```

Graph Representation (Adjacency Matrix)

- Adjacency Matrix
 - We represent the graph in 2D array the size of array for N vertex would be $N \times N$
 - Cell[i,j] will store the weight of the path between vertex i and j.
 - Easier to implement, not suitable for large graph



	KG	JKT	SRG	BKS	CRB	DPK	GS
KG	0	5	-1	5	-1	-1	-1
JKT	5	0	-1	-1	3	-1	-1
SRG	2	-1	0	-1	-1	-1	2
BKS	10	-1	-1	0	3	-1	-1
CRB	-1	3	-1	-1	0	-1	-1
DPK	-1	1	-1	-1	-1	0	-1
GS	-1	2	-1	-1	-1	-1	0



Graph Representation (Adjacency Matrix) (Code)

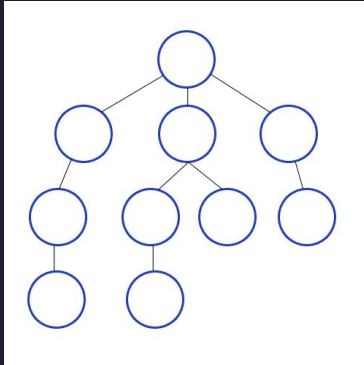
```
void init(int arr[][V]) {  
    int i, j;  
    for (i = 0; i < V; i++)  
        for (j = 0; j < V; j++)  
            arr[i][j] = 0;  
}
```

```
void addEdge(int arr[][V], int i, int j) {  
    arr[i][j] = 1;  
    arr[j][i] = 1;  
}
```

```
int main() {  
    int adjMatrix[V][V];  
  
    init(adjMatrix);  
    addEdge(adjMatrix, 0, 1);  
    addEdge(adjMatrix, 0, 2);  
    addEdge(adjMatrix, 1, 2);  
    addEdge(adjMatrix, 2, 0);  
    addEdge(adjMatrix, 2, 3);  
  
    printAdjMatrix(adjMatrix);  
  
    return 0;  
}
```

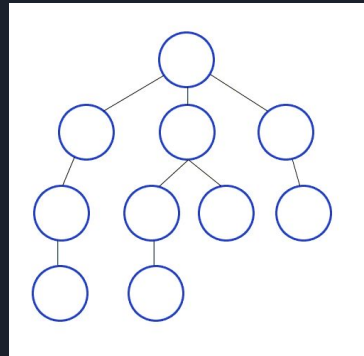
```
void printAdjMatrix(int arr[][V]) {  
    int i, j;  
  
    for (i = 0; i < V; i++) {  
        printf("%d: ", i);  
        for (j = 0; j < V; j++) {  
            printf("%d ", arr[i][j]);  
        }  
        printf("\n");  
    }  
}
```

Graph Traversing (BFS & DFS)



BFS

- Will Branching out to adjacency node first
- Best case would be if the destination is close to the source
- Easiest implementation would be Queue
 - Imagine you push everytime you branching out.
 - Imagine you traverse the next level by popping the queue until empty



DFS

- Will Go deep to one path first till the end of the line.
- Easiest Implementation would be Stack
 - Imagine you push everytime you move forward
 - Imagine you pop everytime you move backward



GRAPH BFS Simulation

- <https://visualgo.net/en/dfsbfbs>



THE END